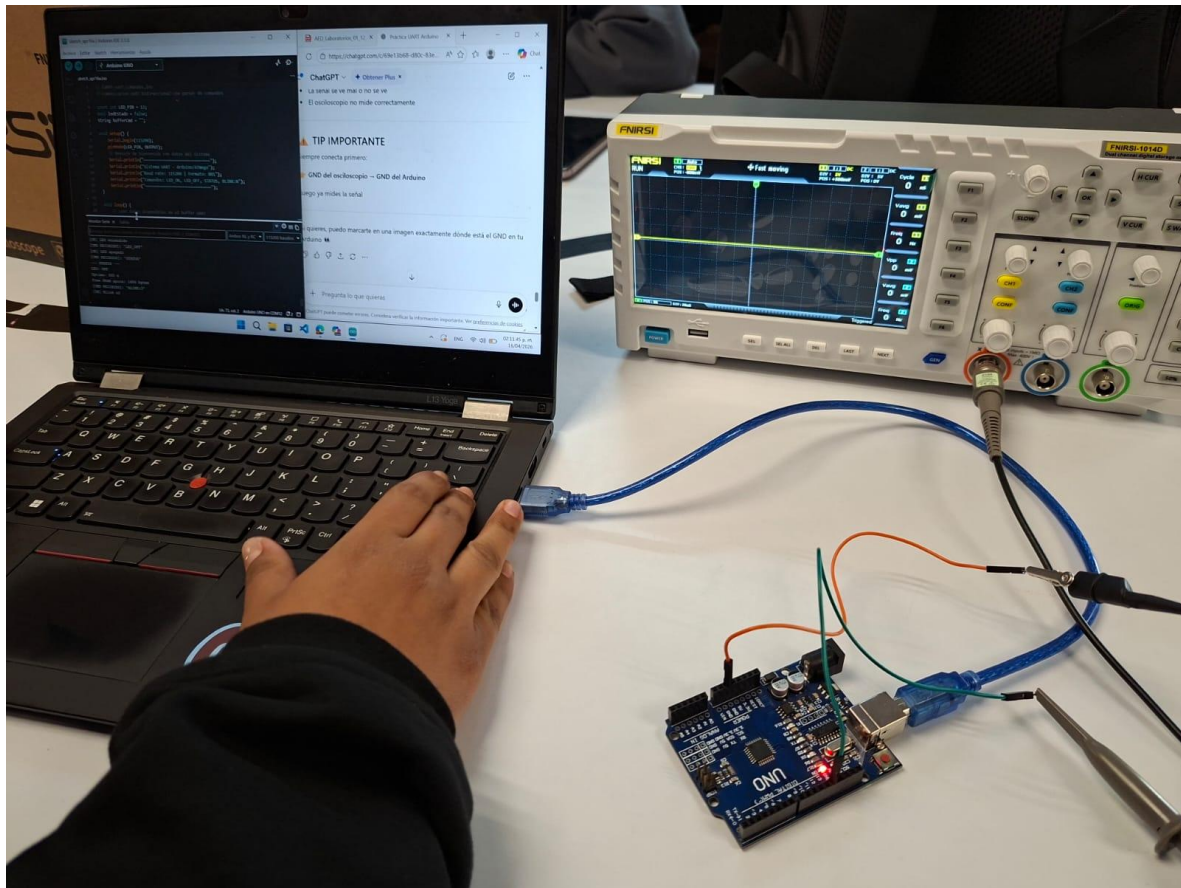


LABORATORIO 05 Comunicación Serial UART: Interfaz entre MCU y PC

Objetivo: Establecer comunicación bidireccional UART entre un microcontrolador y una PC, enviando datos estructurados y procesando comandos recibidos desde el monitor serial.

Materiales:

- Arduino UNO/Nano o ESP32 DevKit
- Cable USB
- Arduino IDE 2.x instalado
- Monitor serial integrado en Arduino IDE (Ctrl+Shift+M)
- Opcional: PuTTY o Termite para comunicación serial avanzada



PREGUNTAS DE REFLEXION Y REPORTE

¿Cuántos bits se transmiten por carácter en formato 8N1 a 115200 bps? ¿Cuánto tiempo tarda en transmitirse la cadena 'LED_ON' completa?

En el formato 8N1, cada carácter se envía usando 10 bits en total: 1 bit de inicio, 8 bits de información y 1 bit de parada.

La cadena 'LED_ON' tiene 6 caracteres, por lo tanto:

$$6 \times 10 = 60 \text{ bits}$$

Si la velocidad de comunicación es de 115200 bits por segundo, transmitir esa cadena tarda aproximadamente 0.52 milisegundos, lo cual es un tiempo muy pequeño. Esto demuestra que la comunicación serial puede ser bastante rápida para enviar comandos simples.

¿Qué ocurre si el Arduino está configurado a 115200 bps pero el monitor serial está a 9600 bps? Prueba este escenario y describe el comportamiento.

Cuando Arduino y el monitor serial están configurados con velocidades diferentes, la comunicación no coincide. En lugar de verse texto normal, aparecen símbolos raros, letras sin sentido o caracteres basura.

Esto pasa porque el monitor serial intenta leer los datos a una velocidad distinta a la que Arduino los está enviando. Para que ambos se entiendan correctamente, siempre deben trabajar con el mismo baud rate.

¿Por qué se usa un buffer de String para acumular los caracteres del comando en lugar de procesar cada byte individualmente al recibirlo?

Se usa un buffer String porque los comandos no llegan completos de una sola vez, sino carácter por carácter. Al guardar cada carácter en un buffer, el programa espera hasta recibir el comando completo y luego lo procesa. Esto hace más fácil reconocer instrucciones completas y evita errores por intentar interpretar datos incompletos.

El comando 'BLINK:N' usa delay() dentro del procesador de comandos. ¿Qué problema tiene esto? ¿Cómo rediseñarías el sistema para que el Arduino pueda recibir otros comandos mientras parpadea?

El problema de usar delay() es que mientras Arduino está esperando, prácticamente se queda ocupado y no puede atender otras tareas. Si se manda otro comando durante ese tiempo, no lo responderá hasta terminar los parpadeos.

Una mejor opción sería usar `millis()`, que permite controlar tiempos sin detener el programa. Así el LED puede seguir parpadeando mientras Arduino continúa revisando si llegan nuevos comandos por el puerto serial.

¿Cómo extenderías el protocolo para que el Arduino envíe datos de sensor cada 2 segundos Y al mismo tiempo procese comandos entrantes? Describe la arquitectura del firmware.

La mejor forma sería organizar el programa para realizar varias tareas al mismo tiempo sin usar pausas bloqueantes.

En el `loop()` se realizan dos tareas:

- Revisar constantemente si llegan comandos por Serial.
- Verificar el tiempo y cada 2 segundos leer el sensor y enviar los datos.

Ejemplo de código:

```
if (millis() - tiempoAnterior >= 2000) {  
    enviarSensor();  
}
```

Con esta estructura, el sistema sería capaz de responder comandos y mandar información del sensor al mismo tiempo, sin interrupciones.